COMPREHENSIVE TECHNICAL & EXAM GUIDE

# คู่มือเจาะลึกระบบคำนวณเงินปันผลประจำปี & แนวคำถาม-คำตอบการสอบโครงงาน

## Annual Dividend Distribution System — Clover Project

คู่มืออธิบายฟังก์ชันการทำงาน ตรรกะทางธุรกิจ สถาปัตยกรรมระบบ โค้ดทั้งฝั่ง Frontend และ Backend พร้อมชุดคำถาม-แนวทางคำตอบครอบคลุมทุกมิติ เพื่อเตรียมความพร้อมสำหรับการสอบป้องกันโครงงานเทคโนโลยีสารสนเทศ

**ระบบสารสนเทศ:** Clover Rubber Cooperative System (Node.js + Express + Prisma ORM + PostgreSQL)
**หน้าที่รับผิดชอบ:** หน้าการจัดการคำนวณเงินปันผล ( `/dividends` )
**จัดทำเอกสารเมื่อ:** ตุลาคม พ.ศ. 2569

---

### สารบัญเนื้อหา (Table of Contents)


1.1 ภาพรวมและวัตถุประสงค์ทางธุรกิจ
1.2 กฎธุรกิจและสูตรการคำนวณเงินปันผล
1.3 สถาปัตยกรรมและโฟลว์การทำงาน 5 ขั้นตอน
1.4 โครงสร้างข้อมูลและฐานข้อมูล (Database Schema)
1.5 เจาะลึกโค้ดฝั่ง Frontend (HTML & JavaScript)
1.6 เจาะลึกโค้ดฝั่ง Backend (API & Data Layer)
1.7 กลไกความปลอดภัยและ Concurrency Control
2.1 คำถามหมวดตรรกะธุรกิจและที่มาของระบบ
2.2 คำถามหมวดความปลอดภัยและ Database Transaction
2.3 คำถามหมวดสถาปัตยกรรมระบบและการรับส่งข้อมูล
2.4 คำถามหมวดเจาะลึกโค้ดและเทคนิคัล
2.5 คำถามหมวดกรณีศึกษาและ Edge Cases


# ส่วนที่ 1: การอธิบายฟังก์ชันการทำงาน ตรรกะ (Logic) แนวคิด (Concept) และโค้ด

## 1.1 ภาพรวมและวัตถุประสงค์ทางธุรกิจ (Business Background & Purpose)

สหกรณ์ชาวสวนยางพาราเป็นองค์กรที่ดำเนินการเพื่อผลประโยชน์ร่วมกันของสมาชิกชาวสวนยาง เมื่อสิ้นสุดรอบปีบัญชีและสหกรณ์มีผลกำไรจากการดำเนินงาน สหกรณ์จะนำผลกำไรส่วนหนึ่งมาจัดสรรเป็น **"เงินปันผลตามส่วนธุรกิจ" (Patronage Dividend)** คืนให้แก่สมาชิกเจ้าของสวนยาง

> **หลักการสำคัญของระบบปันผลใน Clover Project:**
> - **ปันผลตามผลผลิตจริง:** สมาชิกที่ส่งน้ำยางมาก ย่อมได้รับส่วนแบ่งเงินปันผลมากตามสัดส่วนการมีส่วนร่วมทางธุรกิจ
> - **คิดจากน้ำหนักยางแห้ง (Dry Rubber Content):** ไม่คิดจากน้ำหนักน้ำยางสดหรือมูลค่าเงิน เพราะน้ำยางสดมีส่วนผสมของน้ำและความเข้มข้นไม่เท่ากัน น้ำหนักยางแห้งจึงเป็นตัวชี้วัดเนื้อยางที่แท้จริงและยุติธรรมที่สุด
> - **จ่ายตรงเข้ากระเป๋าเงินสะสม (Wallet):** เงินปันผลจะถูกนำไปบวกเพิ่มใน `walletBalance` ของสมาชิกทันท่วงที ทำให้สมาชิกสามารถนำไปเบิกเป็นเงินสดในหน้าเบิกเงินได้ทันที

## 1.2 กฎธุรกิจและสูตรการคำนวณ (Business Rules & Formulas)

ระบบคำนวณเงินปันผลยึดตามข้อกำหนดทางธุรกิจ (Business Constraints) ดังนี้:

**เงินปันผลของสมาชิก (บาท) = น้ำหนักเนื้อยางแห้งรวมตลอดทั้งปี (กก.) × อัตราเงินปันผล (บาท/กก.)**

| ข้อกำหนดธุรกิจ | รายละเอียดและเงื่อนไข | การรองรับในระบบ (System Enforcement) |
|---|---|---|
| **รอบการคำนวณ** | รอบบัญชีประจำปี: **พฤษภาคม ถึง มีนาคม** *(เดือนเมษายนเป็นช่วงปิดกรีด ยางผลัดใบ ไม่มีการรับซื้อ)* | แสดงข้อความกำกับ `DIVIDEND_PERIOD_LABEL = "พฤษภาคม - มีนาคม"` และผูกกับปี พ.ศ. |
| **ความถี่ในการจ่าย** | จ่ายได้ **ปีละ 1 ครั้งเท่านั้น** ห้ามจ่ายซ้ำในปีเดิมเด็ดขาด | - UI ล็อกปุ่มและฟิลด์พร้อมขึ้นแบนเนอร์<br>- Server ตรวจสอบ `prisma.dividendPayment.findFirst`<br>- DB บังคับ `@@unique([memberId, buddhistYear])` |
| **ผู้มีสิทธิ์ได้รับเงิน** | สมาชิกที่มีน้ำหนักยางแห้งสะสมในปีนั้นมากกว่า 0 กิโลกรัม | ฟังก์ชัน `payDividend` กรองเฉพาะสมาชิกที่มี `dryWeightKg > 0` เท่านั้น |
| **การบันทึกยอดเงิน** | ยอดเงินปันผลจะถูกนำไปปรับปรุง 2 ยอด:<br>1. `walletBalance` (ยอดเงินสะสมที่ถอนได้จริง)<br>2. `dividendBalance` (ยอดปันผลสะสมตลอดชีพ) | ประมวลผลผ่าน `tx.member.update` ด้วยคำสั่ง atomic increment `{ increment: amount }` |
| **ห้ามจ่ายปันผลล่วงหน้า** | ไม่สามารถคำนวณหรือจ่ายเงินปันผลของปีในอนาคตได้ | ตรวจสอบทั้ง Client และ Server ห้ามปี พ.ศ. เกิน `getUTCFullYear() + 543` |

## 1.3 สถาปัตยกรรมและโฟลว์การทำงาน 5 ขั้นตอน (5-Step Operational Flow)

1 **โหลดข้อมูลเริ่มต้น (Initial Data Loading)**

เมื่อเปิดหน้าเว็บ ระบบจะเรียก API `GET /api/page-data/dividends` เพื่อดึงรายชื่อสมาชิกทั้งหมด, ข้อมูลน้ำหนักยางแห้งจากการรับซื้อทั้งหมดในอดีต ( `purchases` ), ประวัติการจ่ายปันผล ( `payments` ), และรายการปีที่เคยจ่ายไปแล้ว ( `paidYears` ) นำมาเก็บไว้ใน `state.data`

2 **เลือกปีและกำหนดอัตรา (Parameter Input)**

ผู้ใช้งานเลือกปี พ.ศ. จาก Dropdown (คัดกรองเฉพาะปีที่มีธุรกรรมจริงและไม่เกินปีปัจจุบัน) และกรอกอัตราเงินปันผล (บาท/กก.) เช่น `2.50` จากนั้นกดปุ่ม **"ตกลง"**

3 **จำลองผลการคำนวณฝั่ง Client (Client-Side Simulation & Preview)**

ฟังก์ชัน `render()` ในเบราว์เซอร์จะคำนวณผลลัพธ์ทันที โดยรวมน้ำหนักยางแห้งแยกรายคนผ่าน `Map` และคูณด้วยอัตราปันผล แสดงผลบนการ์ดสรุป 4 ใบ และเรนเดอร์ตารางรายชื่อสมาชิกพร้อมแบ่งหน้า (Pagination 10 รายการ/หน้า) **โดยยังไม่มีการเขียนข้อมูลลง Database ใดๆ ทั้งสิ้น**

4 **ยืนยันการทำรายการผ่าน Modal (Action Confirmation)**

เมื่อตรวจสอบตัวเลขแล้ว ผู้ใช้งานกดปุ่ม **"จ่ายปันผล"** ระบบจะแสดง Modal ป๊อปอัปเตือนอย่างชัดเจนว่า "*เมื่อยืนยันแล้วปันผลจะถูกบวกเพิ่มในยอดเงินสะสมของสมาชิกทันที และจะไม่สามารถแก้ไขหรือคำนวณปันผลปีนี้ซ้ำได้อีก*" เพื่อป้องกันอุบัติเหตุจากการกดผิดพลาด (Human Error)

5 **คำนวณซ้ำและบันทึกผ่าน Transaction ที่ Server (Atomic Execution & System Lock)**

ส่งคำขอ `POST /api/dividends` ไปยังเซิร์ฟเวอร์ เซิร์ฟเวอร์จะ **ไม่เชื่อตัวเลขจาก Client** แต่จะคำนวณใหม่จากฐานข้อมูลทั้งหมด ดำเนินการผ่าน `prisma.$transaction` สร้างรหัสปันผล `D-XXXX` , บันทึกประวัติรายคนพร้อม Snapshot ยอดเงินก่อน-หลัง, บวกเงินเข้า Member Wallet, และบันทึก Audit Log เมื่อสำเร็จ ระบบจะล็อกการจ่ายปันผลของปีนั้นทันที

## 1.4 โครงสร้างข้อมูลและฐานข้อมูล (Database Schema)

ระบบคำนวณเงินปันผลทำงานเชื่อมโยงกับ 3 ตารางหลักใน PostgreSQL ผ่าน Prisma ORM:

| ตาราง (Model) | ฟิลด์สำคัญที่เกี่ยวข้อง | บทบาทหน้าที่ในระบบปันผล |
|---|---|---|
| **Purchase** (การรับซื้อน้ำยาง) | `memberId` (FK) `dryWeightKg` (Decimal) `recordDate` (DateTime UTC) | เป็นแหล่งข้อมูลตั้งต้น (Source of Truth) ของน้ำหนักยางแห้ง ระบบจะกรอง `recordDate` ตามปี พ.ศ. แล้วรวมยอด `dryWeightKg` แยกตาม `memberId` |
| **DividendPayment** (ประวัติการจ่ายปันผล) | `dividendCode` (Unique, D-XXXX) `memberId` (FK) `buddhistYear` (Int) `rate` (Decimal 10,2) `dryWeightKg` (Decimal 10,2) `amount` (Decimal 12,2) `balanceBefore` (Decimal 12,2) `balanceAfter` (Decimal 12,2) `createdAt` (DateTime) | ตารางบันทึกประวัติการจ่ายเงินปันผลรายบุคคลแบบ **Insert-Only** (ห้ามแก้ไข/ห้ามลบ) มี Snapshot ยอดเงินในกระเป๋า `balanceBefore` และ `balanceAfter` เพื่อความโปร่งใส ตรวจสอบย้อนหลังได้ (Audit Trail) |
| **Member** (สมาชิกชาวสวน) | `id` (PK) `walletBalance` (Decimal 12,2) `dividendBalance` (Decimal 12,2) | เมื่อจ่ายปันผล ยอดเงินจะถูกนำไปบวกเพิ่มใน `walletBalance` (สมาชิกสามารถกดเบิกเงินสดได้) และบวกเข้า `dividendBalance` (เก็บบันทึกสถิติเงินปันผลสะสมที่สมาชิกได้รับตลอดชีพ) |

**Database Constraint สำคัญระดับตาราง:**

```
// prisma/schema.prisma
model DividendPayment {
  ...
  @@unique([memberId, buddhistYear]) // สมาชิก 1 คน รับปันผลได้สูงสุด 1 ครั้งต่อปี พ.ศ.
  @@index([memberId])
}
```

Constraint `@@unique([memberId, buddhistYear])` เป็นเกราะป้องกันชั้นเด็ดขาดที่สุดในระดับ Database Engine ต่อให้มี Request ส่งมาพร้อมกัน (Race Condition) Database จะปฏิเสธทันทีด้วย Error Code `P2002`

## 1.5 เจาะลึกโค้ดฝั่ง Frontend (HTML & JavaScript)

### 1) โครงสร้าง UI และองค์ประกอบสำคัญ ( `pages/dividends.html` )

หน้าเว็บแบ่งองค์ประกอบออกเป็น 4 ส่วนหลัก:
**ฟอร์มกำหนดพารามิเตอร์ ( `#dividend-form` ):** มี Dropdown ปี พ.ศ. ( `#year-select` ) และช่องกรอกอัตราปันผล ( `#rate-input` )
**แบนเนอร์ล็อกระบบ ( `#dividend-lock-banner` ):** กล่องสีเหลืองพร้อมไอคอนแม่กุญแจ แสดงข้อความแจ้งเตือนเมื่อปีที่เลือกได้รับการจ่ายปันผลไปแล้ว
**การ์ดสรุปสถิติ 4 ใบ ( `.dividend-stats` ):** น้ำหนักยางแห้งรวม, มูลค่าเงินปันผลรวม, อัตราเงินปันผล, จำนวนสมาชิกทั้งหมด
**ตารางคำนวณและปุ่มจ่ายเงิน ( `#dividend-result` ):** แสดงตารางรายคน, ตัวแบ่งหน้า, และปุ่ม `#pay-button`
**ตารางประวัติการจ่ายปันผล ( `.payout-history` ):** บันทึกรอบการจ่ายในอดีต พร้อมปุ่มเปิด Modal ดูรายละเอียด ( `#payout-modal` )

### 2) การจัดการ State และตรรกะในเบราว์เซอร์ ( `public/js/pages/dividends.js` )

ระบบใช้ State-driven UI Pattern โดยเก็บสถานะไว้ที่ออบเจกต์ `state` ดังนี้:

```
const state = {
  data: null,              // ข้อมูลที่โหลดมาจากเซิร์ฟเวอร์ { members, purchases, paidYears, payments }
  year: null,              // ปี พ.ศ. ที่กำลังเลือกดู
  appliedPeriodLabel: "",  // ช่วงเวลาปันผล เช่น "พฤษภาคม - มีนาคม"
  appliedRate: null,       // อัตราปันผลที่กด "ตกลง" แล้ว (null = ยังไม่คำนวณ)
  error: null,             // ข้อผิดพลาดของการกรอกข้อมูล
  page: 1,                 // หน้าปัจจุบันของตารางปันผล
  paid: false,             // สถานะว่ากดจ่ายสำเร็จในรอบปัจจุบันแล้วหรือไม่
  payError: null,          // ข้อผิดพลาดจากการส่ง API จ่ายเงิน
  selectedPayout: null,    // ข้อมูลประวัติรอบที่กำลังเปิดดู Modal
};
```

### 3) ฟังก์ชันการรวมน้ำหนักยางแห้งด้วย Map (Algorithm Analysis):

```
function dryWeightByMember() {
  const map = new Map();
  for (const purchase of state.data.purchases) {
    if (purchase.buddhistYear !== state.year) continue;
    map.set(purchase.memberId, (map.get(purchase.memberId) ?? 0) + purchase.dryWeightKg);
  }
  return map;
}
```

**ทำไมต้องใช้ Map?:** การใช้ `Map` ทำให้การรวมยอดมีประสิทธิภาพเชิงเวลาเป็น **$O(N)$** (โดย $N$ คือจำนวนการรับซื้อทั้งหมด) และเมื่อต้องการค้นหายอดของสมาชิกในฟังก์ชัน `dividendRows` จะมีความเร็วเป็น **$O(1)$** ต่อคน แทนที่จะต้องใช้ `filter` ซ้อนวนลูปเป็น $O(M \times N)$

#### 4) การเรนเดอร์และควบคุมโหมด Read-Only / Locked:

```
function render() {
  const alreadyPaid = yearAlreadyPaid(); // state.data.paidYears.includes(state.year)
  const paidInfo = alreadyPaid ? getPaidInfoForYear(state.year) : null;

  if (alreadyPaid) {
    lockBanner.hidden = false;
    rateInput.value = formatNumber(state.appliedRate);
    rateInput.disabled = true;   // ปิดช่องกรอก
    calcSubmitBtn.disabled = true; // ปิดปุ่มตกลง
    byId("pay-button").hidden = true; // ซ่อนปุ่มจ่ายเงิน
    byId("pay-locked").hidden = false; // แสดงป้าย "จ่ายเงินปันผลประจำปีนี้เรียบร้อยแล้ว (ล็อกระบบ)"
  } else {
    lockBanner.hidden = true;
    rateInput.disabled = false;
    calcSubmitBtn.disabled = false;
    byId("pay-button").hidden = false;
  }
  ...
}
```

## 1.6 เจาะลึกโค้ดฝั่ง Backend (API & Data Layer)

#### 1) REST API Routing ( `server/routes/dividends.routes.js` )

กำหนดเอนด์พอยต์ `POST /api/dividends` โดยตรวจสอบความครบถ้วนของข้อมูลใน Request Body และเมื่อจ่ายเงินสำเร็จจะเรียก `logActivity` บันทึกลงตาราง `ActivityLog` :

```
await logActivity(req, {
  action: "CREATE_DIVIDEND",
  targetType: "DIVIDEND",
  description: `จ่ายเงินปันผลประจำปี ${body.buddhistYear} (${body.periodLabel}) อัตรา ${body.rate} บาท/กก. ให้สมาชิก ${payments.length} ราย รวม 
});
```

### 2) Data Access Layer และ Transaction ( `server/lib/data/dividends.js` )

หัวใจสำคัญของการจ่ายเงินปันผลอยู่ที่ฟังก์ชัน `payDividend(input)` ซึ่งมีขั้นตอนและมาตรการด้านความปลอดภัยขั้นสูงสุด:

```
export async function payDividend(input) {
  // 1. ตรวจสอบเงื่อนไขพื้นฐาน
  if (!input.rate || input.rate <= 0) throw new DividendError("กรุณากรอกอัตราเงินปันผลเป็นตัวเลขมากกว่า 0");
  const currentBuddhistYear = buddhistYearOf(new Date());
  if (input.buddhistYear > currentBuddhistYear) {
    throw new DividendError(`ยังไม่ถึงปี พ.ศ. ${input.buddhistYear} ในปัจจุบัน ไม่สามารถจ่ายเงินปันผลล่วงหน้าได้`);
  }

  // 2. ตรวจสอบว่าปีนี้เคยจ่ายหรือยัง
  const alreadyPaid = await prisma.dividendPayment.findFirst({
    where: { buddhistYear: input.buddhistYear },
    select: { id: true },
  });
  if (alreadyPaid) throw new DividendError(`ปันผลประจำปี ${input.buddhistYear} ถูกคำนวณและจ่ายไปแล้ว`);

  // 3. ดึงข้อมูลการรับซื้อและคำนวณใหม่ 100% ที่ Server (Zero-Trust Security)
  const purchases = await prisma.purchase.findMany({
    select: { memberId: true, dryWeightKg: true, recordDate: true },
  });
  const dryWeightByMember = new Map();
  for (const purchase of purchases) {
    if (buddhistYearOf(purchase.recordDate) !== input.buddhistYear) continue;
    dryWeightByMember.set(purchase.memberId, (dryWeightByMember.get(purchase.memberId) ?? 0) + purchase.dryWeightKg.toNumber());
  }

  // 4. เริ่ม Atomic Database Transaction
  const payments = await prisma.$transaction(async (tx) => {
    const members = await tx.member.findMany({ where: { id: { in: memberIds } } });
    const created = [];

    for (let i = 0; i < members.length; i++) {
      const member = members[i];
      const dryWeightKg = dryWeightByMember.get(member.id) ?? 0;
      const amount = dryWeightKg * input.rate;
      const balanceBefore = member.walletBalance.toNumber();
      const balanceAfter = balanceBefore + amount;

      // 4.1 สร้างแถว DividendPayment ทีละคน
      const payment = await tx.dividendPayment.create({
        data: {
          dividendCode: `D-${String(startNumber + i).padStart(4, "0")}`,
          memberId: member.id,
          memberCode: member.memberCode,
          memberName: `${member.firstName} ${member.lastName}`,
          buddhistYear: input.buddhistYear,
          periodLabel: input.periodLabel.trim(),
          rate: input.rate,
          dryWeightKg,
          amount,
          balanceBefore,
          balanceAfter,
        },
      });
      created.push(payment);

      // 4.2 ปรับปรุงยอดเงินใน Member (ทั้ง walletBalance และ dividendBalance)
      await tx.member.update({
        where: { id: member.id },
        data: {
          walletBalance: { increment: amount },
          dividendBalance: { increment: amount },
        },
      });
    }
```

```
    return created;
  }).catch((error) => {
    // ดักจับข้อผิดพลาดกรณีจ่ายซ้ำพร้อมกัน (Concurrency / Race Condition)
    if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === "P2002") {
      throw new DividendError(`ปันผลประจำปี ${input.buddhistYear} ถูกจ่ายไปแล้ว หรือมีการจ่ายพร้อมกันจากผู้ใช้อื่น`);
    }
    throw error;
  });

  return payments.map(serializePayment);
}
```

## 1.7 กลไกความปลอดภัยและความถูกต้องของข้อมูลทางการเงิน (Financial Data Integrity)

### สรุป 4 เสาหลักด้านความปลอดภัยทางการเงินของระบบปันผล

Enterprise Grade

1. **หลักการ Zero-Trust (Server-side Recalculation):** Client ส่งมาเพียง `{ buddhistYear, rate, periodLabel }` เท่านั้น ระบบไม่รับตัวเลขยอดเงินรวมหรือน้ำหนักยางแห้งจากเบราว์เซอร์ เพื่อป้องกันการแก้ไขข้อมูลผ่าน Developer Tools
2. **ACID Database Transactions:** การสร้างประวัติการจ่ายของสมาชิกทุกคน และการอัปเดตยอดเงินในกระเป๋าของสมาชิกทุกคน ต้องสำเร็จครบ 100% เท่านั้น หากสมาชิกลำดับสุดท้ายเกิดข้อผิดพลาด ฐานข้อมูลจะทำการ `ROLLBACK` ข้อมูลทั้งหมดกลับสู่สภาพเดิมทันที เงินจะไม่สูญหายหรือเบิกรั่วไหล
3. **Idempotency & Race Condition Prevention:** ป้องกันปัญหาการกดเบิ้ล หรือเปิดสองแท็บแล้วกดพร้อมกัน ด้วยการรวมพลัง 3 ชั้น: (1) UI Disabling (2) Application Logic Check (3) Database Unique Constraint ( `P2002` )
4. **Immutable Audit Trail & Historical Snapshot:** ทุกแถวใน `DividendPayment` บันทึก `balanceBefore` และ `balanceAfter` ไว้ชัดเจน เป็นหลักฐานทางการเงินที่ตรวจสอบย้อนหลังได้โดยไม่ต้องพึ่งพาล็อกภายนอก

# ส่วนที่ 2: รวมแนวคำถาม-คำตอบ สำหรับการสอบป้องกันโครงงาน (Project Defense Q&A)

รวบรวมคำถามที่คณะกรรมการและอาจารย์ผู้สอบนิยมถาม ทั้งเชิงทฤษฎี ตรรกะธุรกิจ สถาปัตยกรรมความปลอดภัย และโค้ดเทคนิค พร้อมแนวทางการตอบแบบมืออาชีพ

## หมวดที่ 1: ตรรกะธุรกิจและที่มาของระบบ (Business Logic & Domain Questions)

### Q1 ทำไมระบบถึงคิดเงินปันผลจาก "น้ำหนักยางแห้ง" (Dry Rubber Content) แทนที่จะคิดจากน้ำหนักน้ำยางสด หรือคิดจากยอดเงินรวมที่สมาชิกขายยางได้?

> **เจตนาของผู้สอบ:** ทดสอบความเข้าใจในบริบทธุรกิจจริง (Domain Knowledge) ของเกษตรกรชาวสวนยางและความยุติธรรมในระบบสหกรณ์

**แนวทางการตอบ:**

เนื่องจากน้ำยางสดที่สมาชิกแต่ละรายนำมาขาย มี **เปอร์เซ็นต์เนื้อยางแห้ง (%DRC)** ไม่เท่ากันขึ้นอยู่กับสภาพสวน สภาพอากาศ และการกรีด เช่น น้ำยางสด 100 กก. ที่มีเนื้อยาง 35% จะได้เนื้อยางจริง 35 กก. ในขณะที่น้ำยางสด 100 กก. ที่มีเนื้อยางเพียง 25% จะมีเนื้อยางจริงเพียง 25 กก.

หากเราคิดจากน้ำหนักน้ำยางสด จะทำให้เกิดความไม่เป็นธรรมต่อสมาชิกที่รักษาคุณภาพน้ำยางเข้มข้น และหากคิดจากยอดเงินขายยาง ราคากลางยางพาราในแต่ละวันมีความผันผวนสูงมาก การใช้ **น้ำหนักยางแห้งสุทธิ ( `dryWeightKg` )** จึงสะท้อนถึง **"ปริมาณผลผลิตเนื้อยางจริง"** ที่สมาชิกส่งมอบให้แก่สหกรณ์ เป็นเกณฑ์สากลของสหกรณ์กองทุนสวนยางที่มีความเที่ยงตรงและยุติธรรมที่สุดครับ

### Q2 เหตุใดรอบการจ่ายเงินปันผลของระบบถึงกำหนดเป็นช่วง "พฤษภาคม - มีนาคม" ทำไมไม่ใช้ปีปฏิทินสากล (1 มกราคม - 31 ธันวาคม)?

> **เจตนาของผู้สอบ:** เช็กว่านักศึกษาลงพื้นที่ศึกษา Business Process จริง หรือแค่นึกขึ้นมาเอง

**แนวทางการตอบ:**

เป็นไปตาม **"รอบฤดูกาลกรีดยางและวงจรบัญชีของสหกรณ์ชาวสวนยาง"** ครับ ในทางปฏิบัติ ต้นยางพาราจะมีการผลัดใบในช่วงฤดูแล้ง คือเดือนมีนาคมถึงเมษายน เกษตรกรจะหยุดกรีดยาง (ปิดหน้ายาง) เพื่อให้ต้นยางพักฟื้น

ดังนั้น ฤดูการกรีดยางจะเริ่มต้นเปิดกรีดในเดือนพฤษภาคม และสิ้นสุดฤดูกาลในเดือนมีนาคมของปีถัดไป สหกรณ์จึงใช้รอบเดือนพฤษภาคมถึงมีนาคมเป็นรอบสรุปผลประกอบการและคำนวณเงินปันผลประจำปี เพื่อให้สอดคล้องกับรายได้และผลผลิตในหนึ่งรอบฤดูกาลผลิตอย่างแท้จริงครับ

### Q3 ในระบบมีทั้ง `walletBalance` และ `dividendBalance` ในตาราง Member ทั้งสองตัวนี้ต่างกันอย่างไร และเงินปันผลไหลไปทางไหน?

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Data Modeling และ Flow ของเงินในระบบ

**แนวทางการตอบ:**

ทั้งสองตัวแปรเก็บข้อมูลยอดเงินคนละวัตถุประสงค์กันครับ:

- `walletBalance` **(กระเป๋าเงินสะสมที่เบิกได้):** เป็นยอดเงินจริงคงเหลือในบัญชีของสมาชิก ซึ่งรวมทั้งรายได้จากการขายยางและเงินปันผล ยอดนี้จะ **เพิ่มขึ้น** เมื่อขายยางหรือได้ปันผล และจะ **ลดลง** เมื่อสมาชิกทำรายการเบิกเงินสดที่หน้า `/withdrawals`
- `dividendBalance` **(สถิติเงินปันผลสะสมตลอดชีพ):** เป็นฟิลด์ที่ทำหน้าที่สะสมยอดเงินปันผลรวมทั้งหมดที่สมาชิกเคยได้รับตั้งแต่สมัครสมาชิก ยอดนี้จะ **เพิ่มขึ้นทางเดียว (Monotonically Increasing)** และไม่ลดลงแม้สมาชิกจะถอนเงินออกไป เพื่อใช้สำหรับแสดงสถิติและผลประโยชน์ตอบแทนสะสมในหน้า Member Portal ครับ

### Q4 ถ้าลูกจ้างกรีดยาง (Employee) กรีดยางในสัญญาจับคู่ (MePair) ลูกจ้างคนนั้นจะได้รับเงินปันผลด้วยหรือไม่?

> **เจตนาของผู้สอบ:** ทดสอบความเข้าใจเรื่องความสัมพันธ์คู่สัญญา (MePair Contract) กับการจ่ายปันผล

**แนวทางการตอบ:**

**ไม่ได้รับครับ เงินปันผลจ่ายให้เฉพาะสมาชิกเจ้าของสวน (Member) เท่านั้น**

เนื่องจากตามหลักการสหกรณ์ เงินปันผลเป็นสิทธิ์ของสมาชิกผู้ถือหุ้นและเจ้าของสวนยาง ส่วนลูกจ้างกรีดยางได้รับส่วนแบ่งรายได้ตามสัญญาจ้าง (เช่น 60% / 40%) ทันที ณ วันที่มีการรับซื้อน้ำยางแต่ละครั้งไปแล้วในบิลรับซื้อ (Purchase) ดังนั้น เงินปันผลประจำปีจึงถูกจัดสรรคืนให้แก่สมาชิกเจ้าของสวนยางผู้เป็นสมาชิกสหกรณ์เท่านั้นครับ

## หมวดที่ 2: ความปลอดภัยและ Database Transaction (Security & Transactions)

### Q5 ทำไมตอนกด "จ่ายปันผล" ฝั่ง Frontend ถึงส่งไปแค่ปี พ.ศ. และอัตราปันผล ทำไมไม่ส่งตารางผลลัพธ์ที่คำนวณแล้วไปให้เซิร์ฟเวอร์บันทึกเลย?

> **เจตนาของผู้สอน:** เช็กความรู้ด้าน Web Application Security และหลักการ Zero-Trust Architecture

**แนวทางการตอบ:**

เป็นการออกแบบตามหลักการ **Zero-Trust Architecture** และ **Server-side Authority** ครับ

หากเราให้ Frontend ส่งยอดเงินปันผลรายคนหรือยอดรวมไป ผู้ใช้งานหรือผู้ไม่ประสงค์ดีอาจใช้ Browser Developer Tools, Postman, หรือดักจับ HTTP Request (Man-in-the-Middle) เพื่อแอบแก้ไขตัวเลขยอดเงินก่อนส่งมาบันทึกได้

ดังนั้น ในระบบของเรา หน้า Frontend มีหน้าที่เพียง **"จำลองการคำนวณ (Simulation Preview)"** ให้ผู้ใช้ตรวจสอบเท่านั้น ส่วนขั้นตอนการจ่ายเงินจริง เซิร์ฟเวอร์จะรับเพียง `buddhistYear` และ `rate` แล้วดึงข้อมูลน้ำหนักยางแห้งจริงจากตาราง `Purchase` ในฐานข้อมูลมาคำนวณใหม่ 100% ทำให้มั่นใจได้ว่าข้อมูลถูกต้องแม่นยำและไม่สามารถถูกแทรกแซงได้ครับ

### Q6 ทำไมต้องใช้ `prisma.$transaction` ในฟังก์ชัน `payDividend` ? ถ้าไม่ใช้แล้วเกิดไฟดับหรือ Server ดับกลางคัน จะเกิดผลกระทบอย่างไร?

> **เจตนาของผู้สอน:** ทดสอบคุณสมบัติ ACID ของ Database (โดยเฉพาะ Atomicity)

**แนวทางการตอบ:**

การจ่ายเงินปันผลประจำปีมีสมาชิกที่เกี่ยวข้องหลายสิบหรือหลายร้อยคน ในการทำงาน 1 ครั้ง มีคำสั่ง Database ย่อยเป็นจำนวนมาก เช่น สมาชิก 50 คน จะต้องมีการสร้าง `DividendPayment` 50 แถว และ `update Member` อีก 50 แถว รวมเป็น 100 การดำเนินการ

หากเรา **ไม่ใช้ Transaction** แล้วเกิดระบบล่มหรือข้อผิดพลาดที่สมาชิกลำดับที่ 25:

- สมาชิก 24 คนแรกจะได้รับเงินและมีประวัติบันทึกไปแล้ว
- สมาชิกคนที่ 25 ถึง 50 จะไม่ได้รับเงิน
- เมื่อเจ้าหน้าที่กลับมากดจ่ายใหม่ ระบบจะปฏิเสธเพราะตรวจพบว่าปีนี้มีการจ่ายไปแล้วบางส่วน ทำให้เกิดปัญหาข้อมูลไม่สอดคล้องกัน (Data Inconsistency) และเงินหายจากระบบ

การใช้ `prisma.$transaction` จะรับประกันคุณสมบัติ **Atomicity (All-or-Nothing)** คือ ต้องทำงานสำเร็จครบทุกรายการ 100% เท่านั้น หากเกิดข้อผิดพลาดแม้แต่จุดเดียว ระบบจะทำการ `ROLLBACK` ข้อมูลทั้งหมดกลับสู่จุดเริ่มต้นเสมือนไม่เคยมีอะไรเกิดขึ้น จึงปลอดภัยต่อข้อมูลทางการเงินสูงสุดครับ

### Q7 ถ้าเจ้าหน้าที่เปิดหน้าคำนวณปันผล 2 เครื่องพร้อมกัน แล้วกดยืนยันจ่ายปันผลปีเดียวกันในเสี้ยววินาทีเดียวกัน (Race Condition) ระบบป้องกันอย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบการรับมือปัญหา Concurrency Control และ Race Condition ในระดับ Database

**แนวทางการตอบ:**

ระบบมีกลไกป้องกัน **Concurrency 3 ระดับ (Multi-Layer Defense)** ครับ:

1. **ระดับ Client UI:** เมื่อกดปุ่มจ่ายปันผล ปุ่มจะเปลี่ยนสถานะเป็น Disabled ทันทีเพื่อกันการกดซ้ำจากหน้าจอเดียว
2. **ระดับ Application Logic:** ฟังก์ชัน `payDividend` จะใช้ `prisma.dividendPayment.findFirst` ตรวจสอบก่อนว่าปีนี้เคยจ่ายหรือไม่
3. **ระดับ Database Constraint (เด็ดขาดที่สุด):** ใน Prisma Schema เราใส่ `@@unique([memberId, buddhistYear])` ไว้ที่ตาราง `DividendPayment` หากมี 2 Request หลุดเข้ามาในเวลาเสี้ยววินาทีเดียวกัน Request แรกจะถูก Commit สำเร็จ ส่วน Request ที่สองจะชนกับ Unique Constraint ทันที ส่งผลให้เกิด Prisma Error `P2002` และโค้ดของเราดักจับข้อผิดพลาดนี้ไว้แล้ว เพื่อคืน Error ข้อความสุภาพว่า "*ปันผลประจำปีถูกจ่ายไปแล้ว หรือมีการจ่ายพร้อมกันจากผู้ใช้อื่น*" โดยไม่มีเงินซ้ำซ้อนแน่นอนครับ

## หมวดที่ 3: สถาปัตยกรรมระบบและการรับส่งข้อมูล (Architecture & Flow)

### Q8 ทำไมหน้าเว็บถึงโหลดข้อมูลผ่าน `loadPageData("dividends")` แทนที่จะยิงเรียก API หลายๆ เส้นแยกกัน เช่น `/api/members`, `/api/purchases`?

> **เจตนาของผู้สอบ:** เช็กแนวคิดเรื่อง Network Performance, Round-trip Time (RTT) และการออกแบบ Page-Data Route

**แนวทางการตอบ:**

เป็นรูปแบบการออกแบบสถาปัตยกรรมแบบ **BFF (Backend-For-Frontend) หรือ Aggregated Page-Data Endpoint** ครับ

หากเราแยกยิง 4 เส้น (Members, Purchases, Payments, PaidYears) เบราว์เซอร์จะต้องเสียเวลาทำ HTTP Handshake ถึง 4 รอบ (เกิด Network Waterfall & High Latency) และหน้าเว็บอาจเกิดอาการกระตุกจากการที่ข้อมูลแต่ละส่วนโหลดเสร็จไม่พร้อมกัน

ระบบจึงรวมศูนย์ไว้ที่ `server/routes/page-data.routes.js` และฝั่งเซิร์ฟเวอร์จะใช้ `Promise.all([ ... ])` ดึงข้อมูลจากฐานข้อมูลพร้อมกันแบบขนาน (Parallel Query) แล้วส่งกลับมาใน JSON Payload ก้อนเดียว ทำให้หน้าเว็บโหลดเร็วกว่ามาก และมั่นใจได้ว่า State เริ่มต้นมีความพร้อมสมบูรณ์ก่อนเรนเดอร์หน้าจอครับ

### Q9 เมื่อเลือกปีที่จ่ายเงินปันผลไปแล้ว หน้าจอจะเปลี่ยนไปอย่างไร และระบบนำข้อมูลจากไหนมาแสดง?

> **เจตนาของผู้สอบ:** ทดสอบความเข้าใจเรื่อง State-driven UI และการแยกโหมดระหว่าง New Calculation กับ Historical View

**แนวทางการตอบ:**

เมื่อเลือกปีที่เคยจ่ายแล้ว (`state.data.paidYears.includes(state.year)`) ระบบจะสลับเข้าสู่ **"โหมดล็อกและดูประวัติ (Locked / Read-Only Mode)"**:

- **แสดงแบนเนอร์ล็อกสีทอง (`#dividend-lock-banner`):** แจ้งว่าการจ่ายเงินปันผลรอบปีนี้เสร็จสิ้นแล้ว และจะเปิดรอบใหม่เมื่อถึงปี พ.ศ. ถัดไป
- **ล็อกฟิลด์อินพุต:** ช่องกรอกอัตราปันผลและปุ่ม "ตกลง" จะถูก Disabled
- **ดึงข้อมูลจริงจากประวัติ:** ฟังก์ชัน `getPaidInfoForYear(state.year)` จะกรองข้อมูลจาก `state.data.payments` เพื่อนำอัตราและยอดเงินที่เคยจ่ายจริงในอดีตมาแสดงบนการ์ดสรุปและตาราง แทนการคำนวณใหม่
- **ซ่อนปุ่มจ่ายเงิน:** ปุ่ม `#pay-button` จะถูกซ่อน และแสดงป้ายสถานะ `#pay-locked` แทน

## หมวดที่ 4: เจาะลึกโค้ดและเทคนิคัล (Code & Database Implementation)

**Q10** **ในฟังก์ชัน `buddhistYearOf(date)` ทำไมต้องใช้ `date.getUTCFullYear() + 543` ทำไมไม่ใช้ `date.getFullYear() + 543` ธรรมดา?**

> **เจตนาของผู้สอบ:** เช็กความรู้เชิงลึกเรื่อง Date Timezone Issue ใน JavaScript/Node.js ซึ่งเป็นบั๊กคลาสสิกของโปรแกรมเมอร์

**แนวทางการตอบ:**

นี่เป็นจุดสำคัญเรื่อง **Timezone Normalization** ครับ!

ในระบบของเรา ฟิลด์ `recordDate` ในตาราง `Purchase` ถูกกำหนดให้เก็บเป็น **"เที่ยงคืน UTC ของวันทำการ"** (เช่น วันที่ 1 มกราคม 2026 จะเก็บเป็น `2026-01-01T00:00:00.000Z`)

หากเราใช้ `date.getFullYear()` แบบ Local Time ถ้าเครื่องเซิร์ฟเวอร์รันอยู่ใน Timezone ที่อยู่ทางตะวันตกของ UTC (เช่น อเมริกา UTC-5) วันที่และเวลาจะถูกเลื่อนถอยหลังไป 5 ชั่วโมง กลายเป็นวันที่ `31 ธันวาคม 2025 เวลา 19:00 น.` ส่งผลให้ `getFullYear()` คืนค่าปี 2025 ทำให้คำนวณปี พ.ศ. ผิดพลาดทันที! การบังคับอ่านผ่าน `getUTCFullYear()` จึงรับประกันความถูกต้องแม่นยำ 100% ไม่ว่าเซิร์ฟเวอร์จะตั้งอยู่บน Cloud ภูมิภาคใดในโลกครับ

**Q11** **ในฟังก์ชัน `payDividend` ทำไมต้องอัปเดตยอดเงินด้วย `{ increment: amount }` แทนที่จะนำยอดเดิมมาบวกใน JavaScript แล้วเขียนทับลงไป?**

> **เจตนาของผู้สอบ:** ตรวจสอบความเข้าใจเรื่อง Atomic Operations ในฐานข้อมูล

**แนวทางการตอบ:**

การใช้ `{ increment: amount }` ของ Prisma จะถูกแปลงเป็นคำสั่ง SQL ระดับฐานข้อมูลโดยตรง เช่น:

```
UPDATE "Member" SET "walletBalance" = "walletBalance" + 1500.00 WHERE "id" = '...';
```

ซึ่งเป็น **Atomic Operation** บนระดับ Database Engine หากเราคำนวณใน JavaScript (เช่น `walletBalance = member.walletBalance + amount`) จะเกิดปัญหา **Lost Update** ได้ง่าย หากในจังหวะนั้นสมาชิกกำลังทำรายการอื่น เช่น มีบิลรับซื้อเข้าหรือเบิกเงินสดพร้อมกัน การคำนวณในหน่วยความจำแล้วเขียนทับค่าเดิมจะไปทับยอดเงินธุรกรรมอื่นให้สูญหายได้ครับ

### Q12 ในภาษา JavaScript ตัวเลขแบบ Floating Point มีปัญหาเรื่องความคลาดเคลื่อน เช่น `0.1 + 0.2 !== 0.3` ระบบนี้จัดการกับปัญหาความคลาดเคลื่อนของทศนิยมทางการเงินอย่างไร?

> **เจตนาของผู้สอบ:** ทดสอบความรู้ด้าน Floating Point Arithmetic & Financial Precision

**แนวทางการตอบ:**

ระบบมีมาตรการจัดการ 2 ส่วนครับ:

- **ในระดับ Database (Prisma / PostgreSQL):** ฟิลด์ทางการเงินทั้งหมดถูกนิยามเป็นชนิดข้อมูล `Decimal(12, 2)` และ `Decimal(10, 2)` ซึ่งจัดเก็บตัวเลขแบบ Exact Numeric (Fixed Point) ไม่ใช้ `Float` หรือ `Double` จึงไม่มีการปัดเศษผิดพลาดในฐานข้อมูลอย่างแน่นอน
- **ในระดับ JavaScript Client:** ใช้ตัวช่วยจัดรูปแบบ `formatNumber()`, `formatCurrency()` จาก `shared/format.js` ซึ่งใช้ `Intl.NumberFormat` ควบคุมทศนิยม 2 ตำแหน่งอย่างแม่นยำ

### Q13 การสร้างรหัสเงินปันผล เช่น `D-0001`, `D-0002` ทำงานอย่างไร ทำไมไม่ใช้ Auto-increment ID ของฐานข้อมูล?

> **เจตนาของผู้สอบ:** เช็กตรรกะการสร้าง Business Document ID (Running Code)

**แนวทางการตอบ:**

ในระบบธุรกิจ เอกสารทางการเงินต้องการ **รหัสเอกสารที่สื่อความหมายและจัดรูปแบบคงที่ (Standard Document Prefix)** เช่น รหัสสมาชิกขึ้นต้นด้วย `M-`, รหัสรับซื้อ `P-`, รหัสเบิกเงิน `W-` และรหัสเงินปันผล `D-` เพื่อให้ง่ายต่อการพิมพ์ลงเอกสารใบเสร็จและง่ายต่อการสื่อสารของเจ้าหน้าที่

ฟังก์ชัน `nextDividendCode()` จะค้นหาแถวล่าสุดที่เรียงตาม `dividendCode DESC` ดึงตัวเลขออกมาแปลงด้วย `parseInt` บวก 1 แล้วจัดรูปแบบด้วย `String(n).padStart(4, "0")` ทำให้ได้รหัสเรียงลำดับอย่างเป็นระเบียบ เช่น `D-0001`, `D-0002` ... `D-9999` ครับ

## หมวดที่ 5: กรณีศึกษาและ Edge Cases (Scenarios & Edge Cases)

### Q14 หากในปีนั้น สมาชิกรายหนึ่งไม่มีการส่งน้ำยางสดเลย (น้ำหนักยางแห้งสะสม = 0 กก.) ระบบจะปฏิบัติต่อสมาชิกคนนี้อย่างไร?

> **เจตนาของผู้สอบ:** ตรวจสอบ Edge Case ของสมาชิกที่ไม่มีผลผลิต

**แนวทางการตอบ:**

ระบบจะจัดการดังนี้ครับ:

- **ในหน้าจำลองคำนวณ (Frontend):** รายชื่อสมาชิกรายนั้นจะยังปรากฏในตาราง เพื่อให้เจ้าหน้าที่ทราบข้อมูลครบถ้วน โดยจะแสดงน้ำหนักยางแห้งเป็น `0.00 กก.` และยอดเงินปันผลเป็น `฿0.00`
- **ในการบันทึกจ่ายเงินจริง (Backend):** ฟังก์ชัน `payDividend` มีการกรองสมาชิกด้วยคำสั่ง:
  `const memberIds = [...dryWeightByMember.keys()].filter(id => dryWeightByMember.get(id) > 0);`
  ดังนั้น สมาชิกที่มียอดเป็น 0 จะ **ไม่ถูกสร้างแถวใน DividendPayment** และไม่มีการบวกยอดเงิน 0 บาทเข้าไป เพื่อลดขยะในฐานข้อมูล (Zero Record Clutter) และประหยัดทรัพยากรการเขียน Disk ครับ

### Q15 ถ้าในอนาคตสหกรณ์มีสมาชิกเพิ่มขึ้นเป็น 10,000 คน ฟังก์ชัน `payDividend` ที่วนลูปบันทึกใน Transaction จะมีปัญหาเรื่อง Performance หรือไม่ และมีแนวทางขยายระบบ (Scalability) อย่างไร?

> **เจตนาของผู้สอบ:** วัดวิสัยทัศน์ทางวิศวกรรมซอฟต์แวร์ (Software Engineering Vision & Scalability)

**แนวทางการตอบ:**

เป็นคำถามที่ยอดเยี่ยมมากครับ! ปัจจุบันระบบได้รับการออกแบบสำหรับสหกรณ์ขนาดกลาง (สมาชิกหลักร้อยถึงพันคน) ซึ่งการวนลูปภายใน Transaction ใช้เวลาเพียงไม่กี่ร้อยมิลลิวินาที

แต่หากในอนาคตมีสมาชิก 10,000 คน การเปิด Transaction ค้างไว้นานอาจทำให้เกิด Database Lock Contention ได้ แนวทางการปรับปรุง (Optimization) ในอนาคตมีดังนี้ครับ:

1. **ใช้ Batch Insertion (`tx.dividendPayment.createMany`):** ส่งคำสั่งสร้างข้อมูลเป็นก้อน ก้อนละ 1,000 แถว แทนการสร้างทีละแถว ซึ่งจะลด Round-trip ใน Database ลงได้มากกว่า 90%
2. **Background Job / Worker Queue:** ใช้ระบบคิว เช่น BullMQ (Redis) เพื่อประมวลผลการจ่ายปันผลในเบื้องหลังแบบ Asynchronous พร้อมแสดง Progress Bar ให้เจ้าหน้าที่ทราบความคืบหน้าแบบ Real-time โดยไม่ทำให้ HTTP Request หมดเวลา (Timeout)
3. **Database Indexing:** เสริม Index บน `recordDate` ในตาราง Purchase และ `buddhistYear` ใน DividendPayment เพื่อให้การค้นหาและสรุปยอดรวดเร็วในระดับมิลลิวินาทีครับ

---